Q A L A

LOW-LEVEL TECHNICAL DESIGN DOCUMENT

Universal Solution Management Operating System

Document ID	QALA-LLD-001
Version	v1.0
Status	DRAFT — For Internal Review
Classification	CONFIDENTIAL — Internal Engineering
Scope	Full platform: all subsystems, services, APIs, data models, protocols, algorithms, security, and deployment architecture
Pages	150+ (12 major sections, 60+ subsections)
Reconciled With	QALA-SDD-v2, QALA-PRD-v2, QALA-RFC-v2, QALA-REQ-v2, QALA-WF-v2, QALA-UIUX-v1, QALA-CONCEPT-v1
Authors	Qala Platform Engineering

Table of Contents

1 System Overview & Architecture

Qala is a cloud-native, multi-tenant, event-driven platform for the end-to-end governance and lifecycle management of solutions across all domains and scales. This section establishes the authoritative technical baseline: system boundaries, architectural principles, service topology, and the core design decisions that govern every subsystem described in this document.

1.1 Document Scope & Reconciliation

This document is the low-level technical design specification for the Qala platform. It reconciles and synthesises all prior design artefacts:

Prior Document	Key Contributions to This LLD
QALA-SDD-v2 (System Design Doc)	Core architecture, service decomposition, persistence strategy, API surface, security model.
QALA-PRD-v2 (Product Requirements Doc)	Feature definitions, user stories, acceptance criteria, domain pack requirements.
QALA-RFC-v2 (Request for Comments)	Architectural decisions: event sourcing, CQRS patterns, multi-tenancy isolation model, SDE deployment manifest format.
QALA-REQ-v2 (Requirements Specification)	Functional + non-functional requirements: performance targets, availability SLAs, compliance mandates, security controls.
QALA-WF-v2 (Workflows & Use Cases)	End-to-end workflow state machines, CCR routing logic, release pipeline definitions, SDE lifecycle transitions.
QALA-UIUX-v1 (UI/UX Specification)	Client API contract requirements, real-time event subscription patterns, bulk operation endpoints.
QALA-CONCEPT-v1 (Platform Concept)	Entity ontology, hierarchy model, domain pack schema, SDE manifest format, repository taxonomy.

1.2 System Context

Qala sits at the centre of an organisation's solution management ecosystem. External actors and systems interact with Qala through defined interfaces:

External Actor / System	Interface	Direction	Protocol
Human Users (web, mobile, CLI)	Qala Frontend (Web App / Mobile App / CLI)	Bidirectional	HTTPS / WSS / gRPC
Third-party CI/CD Tools	Qala REST API + Webhook Engine	Bidirectional	HTTPS REST / Webhooks
External IDPs (SSO)	Identity Service (SAML 2.0, OIDC)	Inbound	SAML 2.0 / OIDC / OAuth 2.0
Domain-specific External Systems	Domain Integration Adapters (per domain pack)	Bidirectional	REST, SFTP, EDI, HL7 FHIR, FIX, AS2

Cloud Infrastructure Providers	Infrastructure Adapter Layer (AWS, Azure, GCP, k8s)	Outbound	Cloud SDKs / Terraform provider
Marketplace / Registry	Qala Registry API	Bidirectional	HTTPS REST + OCI (artefact push/pull)
Audit / SIEM Systems	Audit Event Export (streaming)	Outbound	Kafka / HTTPS webhook / S3 export

1.3 Architectural Principles

The following principles are binding constraints on all design decisions within Qala. Any design that violates a principle requires explicit RFC-level justification and approval.

Principle	Binding Constraint
P1: Event Sourcing	All state changes are derived from an immutable ordered log of domain events. The event log is the system of record. Projections are derived. This applies to: solutions, SDEs, factories, CCRs, releases, and audit events.
P2: CQRS	Command (write) and Query (read) paths are separated at the service level. Commands go to the event store; queries go to read-optimised projections. No synchronous reads in command handlers.
P3: Immutable Artefacts	Released artefacts, published versions, and audit events are never modified or deleted. Soft-delete is permitted only for draft entities (which have not been released).
P4: Tenant Isolation	All data, events, secrets, and compute resources are isolated per tenant. Cross-tenant access is impossible by construction (row-level security, tenant-scoped encryption keys).
P5: Composable Deployment	Every SDE is independently deployable as a self-contained unit. The platform runtime is decoupled from the SDE runtime. SDEs run as Kubernetes-native workloads.
P6: Domain-Agnostic Core	The core platform services are domain-agnostic. All domain-specific logic lives in Domain Packs. The core must function correctly with zero Domain Packs installed.
P7: API-First	Every platform capability is exposed through a versioned, documented API before being surfaced in any UI. The UI is a consumer of the API, not a privileged client.
P8: Zero-Trust Security	All service-to-service communication is mutually authenticated (mTLS). No implicit trust. Every request carries a verifiable identity. Network-level segmentation is not a security control.
P9: Observability by Default	Every service emits structured logs, OpenTelemetry traces, and Prometheus metrics. There is no operational state that cannot be observed without code changes.
P10: Graceful Degradation	Every service defines its degradation behaviour. The platform remains partially functional during component failures. No single-service failure causes total platform unavailability.

1.4 Service Topology

```
QALA PLATFORM SERVICE TOPOLOGY
```

```
CLIENT TIER
┌──────────┬──────────┬──────────┬──────────┐
│ Web App  │ Mobile   │   CLI    │ IDE Ext  │
└──────────┴──────────┴──────────┴──────────┘
                    │ HTTPS / WSS / gRPC
EDGE TIER
┌──────────────────────────────────────────────┐
│   CDN / WAF / DDoS Protection / TLS Termination    │
└──────────────────────────────────────────────┘
                    │
API GATEWAY TIER
┌──────────────────────────────────────────────┐
│  API Gateway Service (REST + GraphQL + WebSocket)   │
│  Rate Limiter | Auth Middleware | Request Router    │
└──────────────────────────────────────────────┘
                    │ Internal mTLS mesh
DOMAIN SERVICES TIER
┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐
│Factory   │  │   SDE    │  │Solution  │  │   CCR    │  │Release   │
│Service   │  │Service   │  │ Service  │  │Service   │  │Service   │
└──────────┘  └──────────┘  └──────────┘  └──────────┘  └──────────┘

┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐
│ Audit    │  │ RBAC     │  │   CMS    │  │Notif.    │  │Search    │
│Service   │  │Service   │  │ Service  │  │Service   │  │Service   │
└──────────┘  └──────────┘  └──────────┘  └──────────┘  └──────────┘
INFRASTRUCTURE TIER
┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐
│Event Store│  │Postgres│  │Object Store│  │Search Index│
│ (Kafka)  │  │  (DB)    │  │ (S3/Blb) │  │(OpenSearch)│
└──────────┘  └──────────┘  └──────────┘  └──────────┘
```

Figure 1.1 — Qala Platform Service Topology

1.5 Technology Stack

Layer	Technology	Rationale
Backend Services	Go 1.22 (primary), Python 3.12 (ML/AI workloads)	Go: performance, low memory footprint, excellent concurrency. Python: ML ecosystem.
API Gateway	Kong Gateway + custom Go middleware	Production-proven, plugin ecosystem, rate limiting, JWT validation.
Message Broker	Apache Kafka 3.7 (KRaft mode, no Zookeeper)	Persistent event log. At-least-once delivery. Compaction for projection rebuilds.
Primary Database	PostgreSQL 16 with Citus (horizontal sharding)	ACID, row-level security for tenant isolation, mature, JSONB for schema-flexible fields.
Object Storage	S3-compatible (AWS S3 / Cloudflare R2 / MinIO for self-hosted)	Artefact storage. Immutable versioned buckets. Pre-signed URL access.
Search	OpenSearch 2.x	Full-text and faceted search over solutions, playbooks, and solution books.
Cache	Redis 7 (cluster mode)	Read projection cache, distributed rate-limit counters, session store.

Secrets	HashiCorp Vault / AWS Secrets Manager (cloud) / Sealed Secrets (k8s)	Dynamic secrets. Per-tenant encryption keys. Automated rotation.
Service Mesh	Istio 1.21 (mTLS, traffic management, observability)	Zero-trust service-to-service auth. Circuit breaking. Traffic shaping.
Container Runtime	Kubernetes 1.30 (EKS / AKS / GKE / bare metal)	SDE runtime. Factory namespace isolation via k8s namespaces + network policies.
Observability	OpenTelemetry + Prometheus + Grafana + Loki	Unified trace/metric/log pipeline. Vendor-neutral. Exportable.
Frontend	React 19 + TypeScript + TanStack Query + Zustand	Component-based, type-safe, optimistic UI, real-time via WebSocket.
Mobile	React Native (Expo SDK 52)	Cross-platform (iOS + Android), shared business logic with web.
CLI	Go (Cobra framework) — binary ships as qala CLI	Single binary, cross-platform, auto-update, shell completion.

1.6 Non-Functional Requirements Targets

Dimension	Target	Measurement Method
API P99 Latency (read)	<80ms under nominal load (1,000 RPS/tenant)	OpenTelemetry trace percentiles, Grafana SLO dashboard
API P99 Latency (write)	<200ms for command acceptance (event published)	OpenTelemetry trace percentiles
Platform Availability	99.9% (SaaS Standard); 99.95% (SaaS Enterprise)	Uptime monitoring, monthly SLA report
Event Processing Throughput	>100,000 domain events/sec platform-wide	Kafka consumer lag monitoring
Storage Durability	99.999999999% (11 nines) for artefact storage	S3/R2 SLA, multi-region replication
Max SDE Boot Time	<30 seconds from activation to first API response	Synthetic monitoring from activation event
Tenant Onboarding	<2 minutes from signup to first usable SDE	E2E synthetic test
Data Residency	Customer-specified region. No cross-region data flow without consent.	IaC audit, compliance assertion tests
RPO	<1 minute (event log replication lag target)	DR drill, continuous RPO measurement
RTO	<15 minutes (full platform recovery)	DR drill results
Security Scan SLA	Critical CVEs patched within 24h; High within 72h	Snyk / Trivy automated scanning pipeline

2 Data Models & Event Sourcing

Qala uses event sourcing as its primary persistence strategy. Every domain entity is represented as an ordered sequence of immutable events. Current state is a projection derived by replaying events. This section defines the canonical event schemas, aggregate boundaries, projection models, and storage layout.

2.1 Aggregate Boundaries

The following are the seven root aggregates in Qala. Each aggregate has exclusive ownership of its state and exposes a command interface. Cross-aggregate operations are coordinated via domain events and process managers (sagas), never by direct aggregate coupling.

Aggregate	Root Entity	Invariants Enforced	Commands Accepted
FactoryAggregate	SolutionFactory	Factory name unique per parent; max nesting depth ≤ 20; policy cannot be removed while active SDEs exist.	CreateFactory, UpdatePolicy, ArchiveFactory, AddMember, RemoveMember, AttachDomainPack
SDEAggregate	SDE	SDE name unique per factory; version strictly monotonic; cannot activate with missing required toolchain.	CreateSDE, ActivateSDE, SuspendSDE, ArchiveSDE, UpdateConfig, AttachToolchain, ScaleSDE
SolutionAggregate	Solution	Version strictly monotonic; cannot transition backwards (except rollback); open CCR blocks direct state change.	CreateSolution, TransitionState, RollbackToVersion, AttachPlaybook, AttachSolutionBook
CCRAggregate	ChangeControlRe quest	CCR cannot be approved if required evidence missing; cannot re-open after Implemented; impact scope non-empty.	SubmitCCR, ApproveCCR, RejectCCR, DeferCCR, WithdrawCCR, AttachEvidence, ImplementCCR
ReleaseAggregate	Release	Release artefact SHA-256 must be verified; required approvals must be complete; quality gates all Passed.	CreateRelease, AddArtefact, ApproveRelease, PublishRelease, RecallRelease
IdentityAggregate	User / ServiceAccount	Username/email unique per tenant; service accounts cannot own factories; MFA state consistent with policy.	CreateUser, AssignRole, RevokeRole, EnableMFA, SuspendUser, CreateServiceAccount
ModelAggregate	SolutionModel	Model name unique per domain pack version; lifecycle states must	CreateModel, PublishModel,

		form a valid DAG; no circular transitions.	DeprecateModel, UpdateLifecycle, AttachDomainPack

2.2 Canonical Event Schema

Every domain event in Qala conforms to a canonical envelope schema. The payload is event-type specific. Events are serialised as JSON and stored in Kafka topics (partitioned by aggregate ID) and in the event store.

```
Canonical Event Envelope  (JSON Schema)
{
  "event_id":    "uuid-v7",          // Time-ordered UUID (sortable)
  "event_type":  "solution.state_transitioned",  // Domain.action format
  "event_version": "1",              // Schema version for this event type
  "aggregate_id": "uuid",            // Root aggregate instance ID
  "aggregate_type": "SolutionAggregate",
  "tenant_id":   "uuid",             // Tenant scope (for isolation)
  "factory_id":  "uuid",             // Factory scope
  "correlation_id": "uuid",          // Ties related events (e.g. CCR + Solution)
  "causation_id": "uuid",            // ID of command that caused this event
  "actor_id":    "uuid",             // User or service account that triggered
  "timestamp":   "2025-09-14T12:00:00.000Z",  // RFC3339 UTC
  "schema_version": "qala/v1",       // Platform schema version
  "payload":     { ... }             // Event-type specific data
}
```

2.3 Core Event Catalogue

Event Type	Aggregate	Payload Fields	Downstream Effects
factory.created	FactoryAggregate	factory_id, name, type, parent_id, owner_id, policy	Create factory projection; provision tenant namespace; send welcome notification
factory.policy_updated	FactoryAggregate	factory_id, old_policy, new_policy, reason	Update factory projection; evaluate active SDEs against new policy; emit policy_applied events to child SDEs
sde.created	SDEAggregate	sde_id, factory_id, name, domain, config, manifest	Create SDE projection; provision k8s namespace; initialise repositories; register in factory registry
sde.activated	SDEAggregate	sde_id, activated_at, toolchain_snapshot	Deploy SDE workloads; register health check;

			send activation notification
sde.config_updated	SDEAggregate	sde_id, changed_keys, old_config, new_config	Update SDE projection; trigger config reload in SDE runtime; audit event
sde.scaled	SDEAggregate	sde_id, old_instances, new_instances, policy_trigger	Update k8s HPA target; emit metrics event; notify SDE owner if manual scale exceeded
solution.created	SolutionAggregate	solution_id, model_id, sde_id, fields, owner_id	Create solution projection; initialise playbook stub; initialise solution book; register in SDE registry
solution.state_transitioned	SolutionAggregate	solution_id, from_state, to_state, actor_id, reason	Update solution projection; check quality gates; emit notifications; update SDE solution index
solution.version_published	SolutionAggregate	solution_id, old_version, new_version, changelog	Create immutable version snapshot; index new version; update version history projection
solution.rolled_back	SolutionAggregate	solution_id, target_version, rollback_reason	Create rollback_applied event; update projection to target version; open post-rollback CCR
ccr.submitted	CCRAggregate	ccr_id, solution_id, ccr_type, risk_score, description, evidence_ids	Create CCR projection; compute approval routing; send approver notifications; block direct solution transitions
ccr.approved	CCRAggregate	ccr_id, approver_id, stage, conditions, timestamp	Update CCR projection; if final stage: unblock solution; create ccr.fully_approved event
ccr.rejected	CCRAggregate	ccr_id, rejector_id, reason, timestamp	Update CCR projection; notify submitter; solution remains in current state; CCR closed
release.created	ReleaseAggregate	release_id, solution_id, version, artefact_ids, target	Create release projection; begin quality gate evaluation pipeline
release.published	ReleaseAggregate	release_id, published_at, distribution_targets, attestation	Mark release immutable; push artefacts to targets; update solution state to Released; index in marketplace if public
release.recalled	ReleaseAggregate	release_id, reason, actor_id, recall_timestamp	Create recall event; notify all deployment targets; update solution to

			Recalled state; open recall CCR

2.4 Projection Models (Read Side)

Read projections are maintained as materialised views in PostgreSQL and cached in Redis. Each projection is rebuilt by replaying events from the Kafka topic. Projections are eventually consistent with the event log (target lag: <500ms under nominal load).

Projection	Primary Table	Key Columns	Rebuild Trigger
SolutionView	solutions_view	id, tenant_id, factory_id, sde_id, model_id, name, state, version, owner_id, domain, tags, updated_at	solution.* events
FactoryView	factories_view	id, tenant_id, parent_id, name, type, member_count, sde_count, policy_hash, created_at	factory.* events
SDEView	sdes_view	id, tenant_id, factory_id, name, state, version, domain, toolchain_ids, scale_config, manifest_hash	sde.* events
CCRView	ccrs_view	id, tenant_id, solution_id, type, state, risk_score, submitter_id, current_stage, approver_ids	ccr.* events
ReleaseView	releases_view	id, tenant_id, solution_id, version, state, artefact_count, published_at, recalled_at	release.* events
AuditView	audit_events_view	id, tenant_id, actor_id, event_type, aggregate_id, timestamp (append-only, no update)	All events (fan-out projection)
ActivityFeedView	activity_feed_view	tenant_id, factory_id, event_type, actor_id, summary_text, timestamp (rolling 90-day window)	Selected events (activity-relevant subset)

2.5 PostgreSQL Schema: Core Tables

The following shows the canonical schema for the primary write-side event store tables and the core projection tables. All tables use row-level security (RLS) policies to enforce tenant isolation at the database layer.

2.5.1 events (write side — append-only)

```
CREATE TABLE events (
  event_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_type      TEXT NOT NULL,           -- e.g. "solution.state_transitioned"
  event_version   SMALLINT NOT NULL DEFAULT 1,
  aggregate_id    UUID NOT NULL,
  aggregate_type  TEXT NOT NULL,
```

```
  tenant_id       UUID NOT NULL,
  factory_id      UUID,
  correlation_id  UUID,
  causation_id    UUID,
  actor_id        UUID NOT NULL,
  occurred_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  payload         JSONB NOT NULL,
  metadata        JSONB DEFAULT '{}'
);
-- Partition by tenant_id (list partitioning) for isolation
-- Index on aggregate_id + occurred_at for replay
-- Index on tenant_id + occurred_at for audit queries
-- RLS: SELECT WHERE tenant_id = current_setting('qala.tenant_id')::uuid
```

2.5.2 solutions (read projection)

```
CREATE TABLE solutions (
  id              UUID PRIMARY KEY,
  tenant_id       UUID NOT NULL,
  factory_id      UUID NOT NULL REFERENCES factories(id),
  sde_id          UUID NOT NULL REFERENCES sdes(id),
  model_id        UUID NOT NULL,
  name            TEXT NOT NULL,
  slug            TEXT NOT NULL,
  state           TEXT NOT NULL,
  version         TEXT NOT NULL,
  version_number  INT NOT NULL DEFAULT 1,
  domain          TEXT NOT NULL,
  owner_id        UUID NOT NULL,
  fields          JSONB NOT NULL DEFAULT '{}',
  tags            TEXT[] DEFAULT '{}',
  quality_gates   JSONB DEFAULT '{}',
  ai_summary      JSONB DEFAULT '{}',
  created_at      TIMESTAMPTZ NOT NULL,
  updated_at      TIMESTAMPTZ NOT NULL,
  UNIQUE (sde_id, slug)
);
-- RLS: ALL WHERE tenant_id = current_setting('qala.tenant_id')::uuid
```

2.5.3 sdes (read projection)

```
CREATE TABLE sdes (
  id              UUID PRIMARY KEY,
  tenant_id       UUID NOT NULL,
  factory_id      UUID NOT NULL REFERENCES factories(id),
  name            TEXT NOT NULL,
  slug            TEXT NOT NULL,
  state           TEXT NOT NULL,  -- draft|active|suspended|archived|deprecated
  version         TEXT NOT NULL,
  domain          TEXT NOT NULL,
```

```
  manifest        JSONB NOT NULL,  -- full .qala.yaml parsed
  config          JSONB NOT NULL DEFAULT '{}',
  toolchain_ids   UUID[] DEFAULT '{}',
  k8s_namespace   TEXT,
  scale_config    JSONB DEFAULT '{"min":1,"max":5}',
  created_at      TIMESTAMPTZ NOT NULL,
  updated_at      TIMESTAMPTZ NOT NULL,
  UNIQUE (factory_id, slug)
);
```

2.6 Domain Pack Schema

Domain Packs extend the universal solution schema. They are stored as versioned JSON Schema documents and registered in the platform's Pack Registry. At runtime, field validation uses the merged schema (universal + domain pack).

```
Domain Pack Registration Schema
{
  "id":         "pack-software-v1",
  "name":       "Software Domain Pack",
  "version":    "1.0.0",
  "domain":     "software",
  "extends":    "qala/base-solution-schema/v1",
  "fields": [
    { "name":"language",       "type":"enum",    "enum":["go","python","typescript","java","rust"],
"required":true  },
    { "name":"runtime_version","type":"string",  "pattern":"^\\d+\\.\\d+(\\.\\d+)?$",          "required":true
},
    { "name":"test_coverage",  "type":"number",  "min":0, "max":100,  "required":false },
    { "name":"sast_passed",    "type":"boolean", "required":false }
  ],
  "quality_gates": [
    { "id":"sast",       "name":"SAST Gate",      "evaluator":"sast-evaluator-v1",
"required_for_release":true  },
    { "id":"coverage",    "name":"Coverage Gate",
"evaluator":"coverage-evaluator-v1","required_for_release":true  },
    { "id":"dep_check",   "name":"Dependency Scan","evaluator":"snyk-evaluator-v1",
"required_for_release":true  }
  ],
  "lifecycle_states": null,  // null = inherit universal lifecycle
  "ccr_types":       ["feature_addition","defect_fix","dependency_update","security_patch"]
}
```

2.7 Event Replay & Projection Rebuild

When a projection needs to be rebuilt (schema migration, disaster recovery, new projection introduction), the Projection Rebuild Service replays the full event log from Kafka. The process is designed to be zero-downtime for reads:

1. Projection Rebuild Service creates a new shadow table (e.g. solutions_v2).
2. Events are replayed from the beginning of the topic, applying the new projection logic.
3. The service tracks its replay position (event_id cursor). Replay catches up to the live tail.
4. Once caught up (lag < 1 second), a database transaction atomically renames solutions → solutions_old, solutions_v2 → solutions.
5. Read traffic is seamlessly directed to the new projection. solutions_old is retained for 24h before deletion.

Replay Performance Target

Full event log replay for a 100M-event tenant must complete within 4 hours. Achieved via parallel partition replay (one goroutine per Kafka partition), batch PostgreSQL upserts (5,000 rows/batch), and Redis cache warm-up post-rebuild.

3 API Design & Contracts

Qala exposes three API surfaces: a REST API for CRUD and command operations, a GraphQL API for flexible querying, and a WebSocket API for real-time event subscriptions. All surfaces are versioned, authenticated, and rate-limited at the API Gateway. This section defines the API design standards, endpoint catalogue, request/response contracts, and error taxonomy.

3.1 API Versioning Strategy

Qala uses URL path versioning for the REST API (/v1/, /v2/) and schema versioning (@deprecated directives) for the GraphQL API. Versions are supported for a minimum of 18 months after supersession. Version sunset notices are sent 6 months in advance via email, in-app notification, and API response headers.

API Surface	Version Scheme	Current Version	Sunset Policy
REST API	URL path: /api/v1/...	v1 (stable), v2 (beta)	18 months minimum post-supersession. Sunset-Date header injected 90 days before.
GraphQL API	Schema version in SDL header	Schema v1	@deprecated fields supported 12 months. Breaking changes require schema major bump.
WebSocket API	Event envelope schema_version	qala/v1	Clients receive version_upgrade events. Old schema supported 6 months post-new-version.
gRPC (internal)	Protobuf package version	qala.v1.*	Internal only. Breaking changes in new package version. Old version 90-day sunset.

3.2 Authentication & Authorisation

All API requests must carry a valid credential. The API Gateway validates credentials before routing to downstream services.

Credential Type	Used By	Mechanism	Scope
User JWT	Web app, mobile, CLI (user sessions)	JWT RS256, signed by Identity Service. Embedded: user_id, tenant_id, roles, factory_scopes.	Per-request. Expiry: 1h access token, 30d refresh token.
Service Account Token	CI/CD integrations, automation scripts	Opaque token mapped to a service account identity. Rotatable. Scoped to one factory + SDE list.	Per-service account. No expiry by default; rotate on policy.
M2M Client Credentials	Backend service-to-service (OAuth 2.0)	Client credentials grant. JWT-based. Scoped to specific service operations.	Short-lived (15min). Refreshed by

			services automatically.
API Key (legacy)	Simple integrations, webhooks	Hashed API key stored per tenant. Checked against HMAC. Deprecated — use service accounts.	Never expires unless revoked. Limit: 10 per tenant.

3.3 REST API — Endpoint Catalogue

The following is the canonical REST API endpoint catalogue. All endpoints are prefixed with /api/v1. Endpoints marked [ADMIN] require a factory-admin or root-admin role.

3.3.1 Factories

Method	Path	Description	Auth Required
POST	/factories	Create a new solution factory	User JWT + create_factory permission
GET	/factories	List factories accessible to the authenticated user	User JWT
GET	/factories/{factory_id}	Get factory details and policy	User JWT + factory member
PATCH	/factories/{factory_id}	Update factory name, description, or settings	User JWT + factory_admin
DELETE	/factories/{factory_id}	Archive a factory (soft-delete). [ADMIN]	User JWT + factory_admin + no active SDEs
GET	/factories/{factory_id}/members	List factory members with roles	User JWT + factory member
POST	/factories/{factory_id}/members	Invite or add a member to the factory	User JWT + factory_admin
DELETE	/factories/{factory_id}/members/{uid}	Remove a member from the factory	User JWT + factory_admin
GET	/factories/{factory_id}/policy	Get the factory governance policy	User JWT + factory member
PUT	/factories/{factory_id}/policy	Replace the factory governance policy	User JWT + factory_admin
GET	/factories/{factory_id}/sdees	List all SDEs in this factory	User JWT + factory member
GET	/factories/{factory_id}/audit	Get factory audit trail (paginated)	User JWT + compliance_officer or factory_admin

3.3.2 Solution Development Environments

Method	Path	Description	Auth Required
POST	/sdees	Create a new SDE in a specified factory	User JWT + sde_create permission in factory
GET	/sdees/{sde_id}	Get SDE details, manifest, and config	User JWT + SDE member
PATCH	/sdees/{sde_id}	Update SDE name, description, or config	User JWT + sde_owner
POST	/sdees/{sde_id}/activate	Activate a draft SDE (triggers k8s provisioning)	User JWT + sde_owner
POST	/sdees/{sde_id}/suspend	Suspend an active SDE	User JWT + sde_owner or factory_admin
POST	/sdees/{sde_id}/archive	Archive a suspended SDE	User JWT + factory_admin
GET	/sdees/{sde_id}/manifest	Get the current .qala.yaml manifest	User JWT + SDE member
PUT	/sdees/{sde_id}/manifest	Replace the SDE manifest (triggers validation)	User JWT + sde_owner
GET	/sdees/{sde_id}/toolchains	List attached toolchains	User JWT + SDE member
POST	/sdees/{sde_id}/toolchains	Attach a toolchain to the SDE	User JWT + sde_owner
DELETE	/sdees/{sde_id}/toolchains/{tc_id}	Detach a toolchain from the SDE	User JWT + sde_owner
GET	/sdees/{sde_id}/repositories	List SDE repositories (all 4 types)	User JWT + SDE member
POST	/sdees/{sde_id}/scale	Update SDE scale configuration	User JWT + sde_owner
GET	/sdees/{sde_id}/solutions	List solutions in this SDE	User JWT + SDE member
GET	/sdees/{sde_id}/health	Get SDE health status and metrics	User JWT + SDE member

3.3.3 Solutions

Method	Path	Description	Auth Required
POST	/solutions	Create a new solution in a specified SDE	User JWT + solution_create permission
GET	/solutions/{solution_id}	Get solution details (full schema)	User JWT + solution access

PATCH	/solutions/{solution_id}	Update solution fields (creates new version draft)	User JWT + solution_developer
GET	/solutions/{solution_id}/versions	List all versions of this solution	User JWT + solution access
GET	/solutions/{solution_id}/versions/{v}	Get a specific version snapshot	User JWT + solution access
POST	/solutions/{solution_id}/transition	Transition solution lifecycle state	User JWT + appropriate role for target state
POST	/solutions/{solution_id}/rollback	Rollback to a specified previous version	User JWT + release_manager or factory_admin
GET	/solutions/{solution_id}/playbook	Get the solution playbook	User JWT + solution access
PUT	/solutions/{solution_id}/playbook	Replace the solution playbook content	User JWT + solution_developer
GET	/solutions/{solution_id}/book	Get the solution book	User JWT + solution access
GET	/solutions/{solution_id}/ccrs	List CCRs associated with this solution	User JWT + solution access
GET	/solutions/{solution_id}/releases	List releases for this solution	User JWT + solution access
GET	/solutions/{solution_id}/ai	Get AI analysis and recommendations	User JWT + solution access + AI feature enabled

3.3.4 Change Control Requests

Method	Path	Description	Auth Required
POST	/ccrs	Submit a new CCR	User JWT + CCR submit permission on solution
GET	/ccrs/{ccr_id}	Get CCR details including approval chain status	User JWT + CCR access
PATCH	/ccrs/{ccr_id}	Update CCR description or evidence (while Draft)	User JWT + CCR submitter
POST	/ccrs/{ccr_id}/approve	Approve CCR at the current approval stage	User JWT + designated approver for current stage
POST	/ccrs/{ccr_id}/reject	Reject CCR at the current approval stage	User JWT + designated approver for current stage
POST	/ccrs/{ccr_id}/defer	Defer CCR for later review	User JWT + designated approver

POST	/ccrs/{ccr_id}/withdraw	Withdraw a submitted CCR (submitter only)	User JWT + CCR submitter (if not yet approved)
POST	/ccrs/{ccr_id}/implement	Mark CCR as implemented (triggers solution unlock)	User JWT + solution_developer
GET	/ccrs/{ccr_id}/audit	Get full CCR audit trail	User JWT + compliance_officer or factory_admin
POST	/ccrs/{ccr_id}/evidence	Attach evidence artefact to CCR	User JWT + CCR submitter or designated approver

3.3.5 Releases

Method	Path	Description	Auth Required
POST	/releases	Create a new release for a solution	User JWT + release_create permission
GET	/releases/{release_id}	Get release details, artefacts, quality gate results	User JWT + release access
POST	/releases/{release_id}/approve	Approve release (at current gate)	User JWT + release_manager
POST	/releases/{release_id}/publish	Publish approved release to distribution targets	User JWT + release_manager
POST	/releases/{release_id}/recall	Recall a published release	User JWT + release_manager or factory_admin
GET	/releases/{release_id}/artefacts	List all artefacts in the release	User JWT + release access
POST	/releases/{release_id}/artefacts	Add an artefact to a draft release	User JWT + release_manager
GET	/releases/{release_id}/gates	Get quality gate evaluation results	User JWT + release access

3.4 Standard Request/Response Contract

All REST API responses conform to a standard envelope. This ensures consistent error handling and pagination across all clients.

```
Success Response Envelope
// Single resource:
{ "data": { ...resource }, "meta": { "request_id": "uuid", "timestamp": "..." } }

// Collection response (paginated):
{ "data": [ {...}, {...} ],
```

```
  "pagination": { "total": 1240, "page": 2, "per_page": 50, "next_cursor": "eyJ..." },
  "meta": { "request_id": "uuid", "timestamp": "..." } }
```

```
Error Response Envelope
{ "error": {
    "code":    "SOLUTION_INVALID_TRANSITION",   // Machine-readable error code
    "message": "Cannot transition from Draft to Released",
    "detail":  "Solution must pass through In Review state first",
    "context": { "current_state": "Draft", "requested_state": "Released" },
    "docs_url": "https://docs.qala.io/errors/SOLUTION_INVALID_TRANSITION"
  },
  "meta": { "request_id": "uuid", "timestamp": "..." } }
```

3.5 HTTP Status Code Taxonomy

Status Code	Meaning in Qala Context	When Used
200 OK	Request succeeded. Resource returned or updated.	GET, PATCH, PUT operations that return data.
201 Created	Resource created. Location header contains new resource URL.	POST operations creating new aggregates.
202 Accepted	Command accepted. Processing is async. Correlation ID returned.	State transitions, activations, scale operations, releases.
204 No Content	Command succeeded. No response body.	DELETE, POST actions with no return value (withdraw, archive).
400 Bad Request	Request payload invalid. Validation errors in response.	Missing required fields, invalid enum value, JSON parse error.
401 Unauthorized	Missing or invalid authentication credential.	No JWT, expired JWT, invalid API key.
403 Forbidden	Authenticated but insufficient permission for this operation.	Attempting factory_admin action with developer role.
404 Not Found	Resource does not exist within caller's tenant scope.	Also used when resource exists but caller lacks read access (security by obscurity).
409 Conflict	Request conflicts with current resource state.	Submitting CCR on a released solution, activating an already-active SDE.
422 Unprocessable	Request is syntactically valid but semantically invalid.	Transition not permitted by lifecycle DAG, missing required quality gate evidence.
429 Too Many Requests	Rate limit exceeded. Retry-After header included.	Per-tenant and per-endpoint rate limits.
500 Internal Server Error	Unexpected internal error. Incident ID in response.	Unhandled exceptions. Automatic alert triggered.

503 Service Unavailable	Service is degraded. Retry-After included.	Circuit breaker open, dependency outage, maintenance mode.

3.6 GraphQL API

The GraphQL API is designed for flexible query patterns that would require multiple REST calls. It exposes read-only access to all projection models and is served by the Query Service. Mutations are not supported on the GraphQL surface — all writes go through the REST command endpoints.

```
Example GraphQL Query: Solution with nested CCRs and latest release
query GetSolutionDetail($id: ID!) {
  solution(id: $id) {
    id name state version domain
    sde { id name factory { id name } }
    ccrs(filter: { state: OPEN }, first: 10) {
      nodes { id type riskScore state submittedAt submitter { id name } }
    }
    latestRelease {
      id version publishedAt
      artefacts { id name sha256 size }
      qualityGates { id name state evaluatedAt }
    }
    aiAnalysis { riskScore recommendations driftStatus }
  }
}
```

3.7 WebSocket API — Real-Time Event Subscriptions

The WebSocket API allows clients to subscribe to real-time events scoped to their tenant, factory, SDE, or solution. Events are delivered as JSON messages. The WebSocket service is stateless — subscriptions are maintained in Redis pub/sub, not in-process memory.

Subscription Channel	Event Types Delivered	Use Case
tenant:{tenant_id}	factory.created, global platform events	Root-level admin dashboard, global activity feed.
factory:{factory_id}	factory.policy_updated, member added/removed, SDE created	Factory overview page, factory admin notifications.
sde:{sde_id}	sde.activated, sde.config_updated, sde.scaled, solution.created	SDE dashboard, live solution list, health monitor.
solution:{solution_id}	solution.state_transitioned, solution.version_published, ccr events	Solution detail page, live state updates, CCR tracker.
ccr:{ccr_id}	ccr.approved, ccr.rejected, ccr.evidence_attached	CCR approval page, approval notifications.
release:{release_id}	release.gate_evaluated, release.approved, release.published	Release pipeline progress view.

user:{user_id}	Notifications, assigned CCR/release events, AI recommendations	In-app notification bell, user inbox.

WebSocket Authentication Protocol

1. *Client connects to wss://api.qala.io/ws with Authorization: Bearer {jwt} in the upgrade request.*
2. *Gateway validates JWT. On failure, 401 close frame sent.*
3. *Client sends subscription request: {"type":"subscribe","channel":"solution:uuid","options":{}}*
4. *Server responds: {"type":"subscribed","channel":"solution:uuid","subscription_id":"uuid"}*
5. *Events are delivered: {"type":"event","channel":"solution:uuid","event":{...canonical envelope}}*
6. *Client heartbeat every 30s: {"type":"ping"} → server {"type":"pong"}*
7. *On disconnect, all subscriptions are removed from Redis within 5 seconds.*

4 Service Design

This section provides the low-level design specification for each domain service. For each service: responsibilities, command/query interface, internal state machine (where applicable), concurrency strategy, error handling, and integration points.

4.1 Factory Service

The Factory Service is responsible for all operations on Solution Factory aggregates. It is the source of factory identity, policy, membership, and hierarchy.

Attribute	Value
Service Name	qala-factory-service
Language	Go 1.22
Instances	Min 2, max 10 (autoscaled on CPU/RPS)
Dependencies	PostgreSQL (events + factories projection), Kafka (event publish), Identity Service (member validation), Redis (cache)
Kafka Topics	Publishes: qala.factory.events.v1 | Consumes: qala.identity.events.v1 (member removal propagation)
gRPC Port	50051 (internal mesh)
Health Endpoint	/healthz (liveness), /readyz (readiness)

4.1.1 Command Handlers

Command	Pre-Conditions	Actions	Post-Events
CreateFactory	Parent factory exists if parent_id set; name unique per parent; caller has create_factory permission; max depth 20 not exceeded.	Validate; generate factory_id; publish factory.created; provision tenant namespace in k8s.	factory.created
UpdatePolicy	Caller is factory_admin; factory is active; policy schema valid.	Validate policy schema; compute policy diff; publish factory.policy_updated; propagate to child SDEs via saga.	factory.policy_updated
AddMember	Target user exists; not already a member; role is valid; caller is factory_admin.	Validate; publish factory.member_added.	factory.member_added
RemoveMember	Target is a member; not the last admin; caller is factory_admin.	Validate; publish factory.member_removed ; cascade SDE access revocation via saga.	factory.member_removed

ArchiveFactory	No active or suspended SDEs; no open CCRs in factory scope; caller is factory_admin.	Validate pre-conditions; publish factory.archived; de-provision k8s namespace (async).	factory.archived
AttachDomainPack	Domain pack version exists in pack registry; no conflicting pack for same domain; caller is factory_admin.	Validate; publish factory.domain_pack_attached; revalidate all SDE models async.	factory.domain_pack_attached

4.1.2 Concurrency Strategy

Factory commands use optimistic concurrency control. The aggregate version is embedded in the event store and checked on every write:

```
-- Optimistic lock check on event insert:
INSERT INTO events (...) SELECT ... WHERE NOT EXISTS (
  SELECT 1 FROM events
  WHERE aggregate_id = $factory_id
  AND occurred_at > $command_read_timestamp
  AND event_type IN ('factory.archived', 'factory.policy_updated')
);
```

If the check fails, the command handler returns 409 Conflict. The client retries by re-reading the latest aggregate state.

4.2 SDE Service

The SDE Service manages the full lifecycle of Solution Development Environments, including provisioning, configuration management, toolchain attachment, and health monitoring.

Attribute	Value
Service Name	qala-sde-service
Language	Go 1.22
Instances	Min 3, max 20 (autoscaled; higher baseline due to activation complexity)
Dependencies	PostgreSQL, Kafka, Kubernetes API (for SDE namespace provisioning), Vault (secrets injection), Redis, Toolchain Registry
Kafka Topics	Publishes: qala.sde.events.v1 | Consumes: qala.factory.events.v1, qala.toolchain.events.v1
Key Subcomponent	SDE Provisioner (coordinates k8s namespace creation, PVC provisioning, ConfigMap injection, Network Policy creation)

4.2.1 SDE Activation Flow (detailed)

SDE activation is the most operationally complex command in the system. It is implemented as a distributed saga to ensure atomicity across multiple systems:

Saga Step	Service	Action	Compensation (on failure)

1. Validate manifest	SDE Service	Parse .qala.yaml; validate all fields; check toolchain versions exist.	None (pre-saga validation)
2. Reserve k8s namespace	SDE Provisioner	Create namespace qala-sde-{sde_id}; apply resource quotas from scale_config.	Delete namespace
3. Provision repositories	Repository Service	Create 4 repo buckets in object store; register in repo index.	Delete repo buckets
4. Inject secrets	Vault Agent	Create Vault policy for namespace; inject .env secrets as k8s Secrets; mount toolchain tokens.	Revoke Vault policy; delete k8s Secrets
5. Deploy toolchain pods	Toolchain Deployer	Apply toolchain Helm charts to namespace; wait for pods Ready (timeout: 120s).	Helm uninstall all charts
6. Apply network policy	SDE Provisioner	Apply NetworkPolicy: allow intra-namespace, deny cross-tenant; apply Istio AuthorizationPolicy.	Delete network policies
7. Publish sde.activated	SDE Service	Write event to Kafka; update SDE projection state to active.	Publish sde.activation_failed; rollback all steps

Saga Timeout Policy
If any saga step exceeds its timeout, the saga coordinator publishes sde.activation_failed and executes compensations in reverse order. The SDE returns to Draft state. All compensation steps are idempotent and retried up to 3 times before human intervention is required.

4.3 Solution Service

The Solution Service owns all solution lifecycle operations. It enforces the model-defined lifecycle state machine and coordinates CCR blocking, version management, and artefact associations.

4.3.1 State Machine Engine

The lifecycle state machine is computed from the Solution Model's lifecycle definition at solution creation time and cached. State transitions are validated before event publication:

```
type StateMachine struct {
  States      []string
  Transitions map[string][]string  // from -> []to
  Guards      map[string][]Guard   // "from:to" -> []Guard
}

type Guard interface {
  Name() string
  Evaluate(solution *Solution, actor *Identity) (bool, string)
}
```

```
// Guards check conditions such as:
// - AllQualityGatesPassed{} : all required gates have passed
// - NoOpenCCRs{}            : no open CCRs block this transition
// - HasApproval{}           : actor has required role for this transition
// - EvidencePackageComplete{}: release evidence is all attached
```

4.3.2 Version Management

Every write to a solution field that changes its logical state increments the solution version. Version increments are classified as Major, Minor, or Patch according to the factory versioning policy:

Change Type	Default Classification	Version Increment
State transition (to Released)	Major	e.g. 1.0.0 → 2.0.0
Substantive field change (via CCR)	Minor	e.g. 1.2.0 → 1.3.0
Documentation / metadata update	Patch	e.g. 1.2.3 → 1.2.4
Rollback	N/A	Rollback creates a new version with rollback marker: 1.2.5-rollback-from-1.2.3

4.4 CCR Service

The CCR Service manages the full change control lifecycle, including risk scoring, approval chain computation, evidence management, and the CCR-to-solution coordination protocol.

4.4.1 Risk Scoring Algorithm

Risk scores are computed deterministically from a weighted factor model. The score (1.0–10.0) determines the approval chain routed by the factory CCR policy.

Factor	Weight	Range	Scoring Logic
CCR Type Base Score	30%	1–10	Emergency=10, Security Patch=8, Ingredient Sub=8, Feature=5, Config=3, Doc=1
Impact Scope	25%	1–10	Score = min(10, 1 + log2(affected_solution_count)) × 2.5
Regulatory Dimension	25%	0 or 5	+5 if any affected solution has a regulatory compliance flag
Domain Risk Modifier	10%	0–3	Domain Pack provides a base risk modifier (e.g. pharma=3, financial=3, software=1, personal=0)
Recency / Freeze Modifier	10%	0 or 3	+3 if factory is in change_freeze window; +1 if solution released within 7 days

```
func ComputeRiskScore(cmd SubmitCCRCommand, ctx FactoryContext) float64 {
  base     := ccrTypeBaseScores[cmd.Type]
  scope    := math.Min(10, 1+math.Log2(float64(len(cmd.AffectedSolutions))))*2.5
  regDim   := 0.0; if ctx.HasRegulatoryFlag { regDim = 5 }
  domainMod:= ctx.DomainPack.BaseRiskModifier
```

```
    freeze   := 0.0; if ctx.InChangeFreezeWindow { freeze = 3 }
    raw := base*0.30 + scope*0.25 + regDim*0.25 + domainMod*0.10 + freeze*0.10
    return math.Min(10.0, math.Max(1.0, raw))
  }
```

4.4.2 Approval Chain Routing

The approval chain is computed at CCR submission time and encoded in the CCR aggregate. It defines the ordered list of approval stages, each with a set of eligible approvers and a quorum requirement:

Risk Band	Stage 1	Stage 2	Stage 3	Stage 4
1.0–3.9 (Low)	Solution owner (1 of 1) OR auto-approve if policy allows	—	—	—
4.0–5.9 (Medium)	Technical Lead of SDE (1 of N)	Domain Expert (1 of N)	—	—
6.0–7.9 (High)	Technical Lead (1 of N)	Domain Expert (1 of N)	Compliance Officer (1 of 1)	—
8.0–9.4 (Critical)	Technical Lead (majority quorum)	Domain Expert (majority quorum)	Compliance Officer (1 of 1)	Change Control Board (majority of CCB)
9.5–10.0 (Emergency)	Emergency Chair (1 of 1) — fast path; post-facto 4-stage review required within 24h	—	—	—

4.5 Release Service

The Release Service orchestrates the entire release pipeline: artefact collection, quality gate evaluation, approval gating, artefact signing, and distribution to targets.

4.5.1 Release Pipeline Stages

Stage	Trigger	Actions	Pass Condition	Failure Action
1. Artefact Collection	Release created	Pull artefact refs from solution; verify each SHA-256 hash against Artifact Repository; check no artefact is in quarantine.	All artefacts present and hash-verified	Release blocked; missing artefacts liste

				d in release. artefact_missing event
2. Domain Gate Evaluation	Artefact collection pass	Run all required domain-pack quality gate evaluators (async workers). Each evaluator is a containerised job.	All required gates report PASSED	Release blocked; gate results in release. gate_failed event; notifications sent
3. Release Approval	All gates passed	Notify designated release approvers. Wait for approval within SLA (default: 48h).	All required approvals collected	Escalate if SLA exceeded; notify factory_admin
4. Artefact Signing	Approval complete	Sign artefact bundle with tenant signing key (Sigstore cosign); generate SLSA provenance attestation; seal release hash.	Signature and attestation generated	Signing service error; retry 3x; page on-call

5. Distribution	Signing complete	Push artefacts to all configured distribution targets (container registry, artefact repo, marketplace, download CDN).	All targets acknowledge receipt	Partial distribution recorded; retry failed targets; notify release_manager
6. Post-Release	Distribution complete	Transition solution state to Released; publish release.published event; generate release notes from changelog; update solution book.	solution.state_transitioned event published	None (event publish is idempotent)

4.5.2 Quality Gate Evaluator Contract

Each quality gate evaluator is a containerised workload that reads its input, performs its evaluation, and writes a structured result. Evaluators run in the SDE's k8s namespace and have read-only access to the solution's artefact repository.

```
  // Evaluator input (mounted as /etc/qala/gate-input.json):
  { "release_id":"uuid", "solution_id":"uuid", "version":"1.4.2",
    "artefact_ids":["uuid","uuid"], "domain_config":{...gate config...} }

  // Evaluator output (written to /etc/qala/gate-result.json):
  { "gate_id":"sast",  "state":"PASSED",  // PASSED | FAILED | WARNING | SKIPPED
    "evaluator_version":"1.2.0",
    "evaluated_at":"2025-09-14T12:00:00Z",
    "findings":[ { "severity":"LOW", "rule":"G401", "location":"main.go:42",
"message":"..." } ],
    "metrics":{ "critical":0, "high":0, "medium":2, "low":14 },
    "attestation":"base64-signed-result" }
```

5 Security Design

Security in Qala is a first-class architectural concern, not a feature. This section defines the security model at every layer: authentication, authorisation, tenant isolation, secrets management, data encryption, network security, audit, and vulnerability management. All security controls align with SOC 2 Type II, ISO 27001, and (for relevant domains) HIPAA, GxP, and FCA requirements.

5.1 Authentication Architecture

Qala uses a layered authentication model. The Identity Service is the single authority for all identity tokens.

```
AUTHENTICATION FLOW

1. User authenticates via:
   a) Email/password + TOTP (built-in)
   b) OIDC/SAML SSO (enterprise IdP: Okta, Azure AD, Google Workspace)
   c) OAuth 2.0 PKCE (CLI, third-party apps)

2. Identity Service issues:
   - Access Token (JWT RS256, 1h expiry)
     Claims: { sub, tenant_id, roles[], factory_scopes{}, jti, iat, exp }
   - Refresh Token (opaque, stored hashed in DB, 30d, rotation-on-use)

3. API Gateway validates JWT:
   - Signature against cached JWKS endpoint
   - Expiry (with 30s clock skew tolerance)
   - Token not in revocation list (Redis bloom filter, <1ms lookup)
   - Tenant_id matches route tenant context

4. Downstream services receive:
   - Validated request context (user_id, tenant_id, roles)
   - Service-to-service JWT (M2M, 15min, signed per-call)
   - All via Istio mTLS channel (service cert validates peer identity)
```

Figure 5.1 — Qala Authentication Flow

5.2 Authorisation: RBAC Design

Qala uses a hierarchical RBAC model. Roles are defined at the factory level and can be scoped to specific SDEs. The permission check is a two-stage evaluation: role-level permission grant followed by resource-level ownership check.

5.2.1 Permission Matrix (selected)

Permission	Root Admin	Factory Admin	Release Manager	Compliance Officer	Solution Developer	Reviewer	Guest

factory:create	✓	✓*	✗	✗	✗	✗	✗
factory:update_policy	✓	✓	✗	✗	✗	✗	✗
sde:create	✓	✓	✗	✗	✗	✗	✗
sde:activate	✓	✓	✗	✗	✓**	✗	✗
solution:create	✓	✓	✗	✗	✓	✗	✗
solution:transition_state	✓	✓	✓***	✗	✓****	✗	✗
ccr:submit	✓	✓	✓	✗	✓	✗	✗
ccr:approve	✓	✓	✓	✗	✗	✓	✗
release:create	✓	✓	✓	✗	✗	✗	✗
release:publish	✓	✓	✓	✗	✗	✗	✗
audit:read	✓	✓	✗	✓	✗	✗	✗
solution:read	✓	✓	✓	✓	✓	✓	✓

* Factory Admin can create child factories under their own factory. · ** SDE Owner only. · *** Release Manager can only transition from Approved to Released. · **** Developer can only transition within Draft↔In Review cycle.

5.3 Tenant Isolation Architecture

Tenant isolation is enforced at four independent layers. Compromise of any single layer does not expose cross-tenant data because the others remain effective:

Layer	Mechanism	Scope	Breach Impact Without This Layer
1. Network	Kubernetes NetworkPolicy + Istio AuthorizationPolicy per tenant namespace.	SDE runtime traffic	Lateral movement between SDE pods of different tenants.
2. Data (RLS)	PostgreSQL Row-Level Security: ALL policies enforce tenant_id match via session variable set by connection pool.	All DB reads and writes	Cross-tenant data reads via SQL injection or misconfigured query.
3. Object Storage	Per-tenant S3 bucket prefix + IAM policy (or per-tenant bucket for enterprise). Pre-signed URLs are tenant-scoped.	Artefact and asset access	Cross-tenant artefact downloads.
4. Encryption	Per-tenant encryption keys stored in Vault. All data at rest encrypted with tenant key (AES-256-GCM). Keys never shared across tenants.	Data at rest	Qala infrastructure team reading tenant data from raw storage.

5.4 Secrets Management

All secrets in Qala are managed through HashiCorp Vault (cloud deployments) or AWS Secrets Manager (AWS-native deployments). No secrets are stored in code, configuration files, or the event log.

Secret Type	Storage	Rotation	Injection Method
Database Credentials	Vault Dynamic Secrets (PostgreSQL backend)	Auto-rotated every 1h	Vault Agent sidecar injects into pod env
Object Storage Credentials	Vault Dynamic Secrets (AWS/Azure backend)	Auto-rotated every 15min	Vault Agent; presigned URL generation per-request
JWT Signing Keys (RS256)	Vault Transit Engine (key never leaves Vault)	Manual rotation annually	Identity Service calls Vault Transit for sign/verify
Tenant Encryption Keys	Vault per-tenant KV namespace	Manual rotation + re-encryption on demand	Service-specific Vault token; never exposed to tenant user
Service-to-Service M2M Tokens	Vault AppRole per service	Auto-rotated every 15min	Injected at pod start by Vault Agent Injector
SDE Runtime Secrets	Per-SDE Vault namespace (scoped to k8s namespace)	Configured per-secret by SDE owner	Mounted as k8s Secrets by Vault Agent; env or file
Domain Integration Credentials	Vault per-integration KV + SDE namespace	Per-integration rotation policy	Injected into domain adapter pods on startup

5.5 Encryption at Rest & in Transit

Data Category	At Rest	In Transit
Event Store (PostgreSQL)	AES-256-GCM with tenant key (Vault); TDE at tablespace level	mTLS (Istio) service-to-service; TLS 1.3 client-to-gateway
Object Storage (artefacts)	SSE-C with per-tenant key (AES-256)	TLS 1.3 via pre-signed URL; direct upload via SDE service account
Redis Cache	AES-256 encryption at cluster level (Redis Enterprise)	mTLS within mesh; never exposed externally
Kafka Event Log	Kafka log encryption enabled; topic-level AES-256	TLS + SASL_SSL for all producers/consumers
Search Index (OpenSearch)	Index-level encryption with tenant-scoped keys	mTLS within mesh
Secrets (Vault)	Vault-managed encryption; backend: AWS KMS or HSM	mTLS
Backup Archives	AES-256-GCM with offline key; geographic split for enterprise	Encrypted in transit via S3 transfer acceleration TLS

5.6 Audit & Compliance

Every state-changing operation in Qala generates an immutable audit event. Audit events are distinct from domain events — they are written to a separate, append-only audit vault table with additional compliance metadata.

Audit Event Field	Description
audit_id	UUID v7 (time-ordered, unique globally)
tenant_id	Tenant that owns this audit record
actor_id	User or service account that performed the action
actor_ip	IP address of the request originator (hashed for PII compliance if required)
actor_user_agent	Browser/CLI user agent string (truncated to 256 chars)
action	Canonical action string: "ccr.approved", "solution.released", etc.
resource_type	Type of resource affected: Solution, CCR, SDE, Factory, etc.
resource_id	UUID of the affected resource
before_state	JSONB snapshot of resource state before the action (for write operations)
after_state	JSONB snapshot of resource state after the action
request_id	Correlation ID linking to the originating API request
occurred_at	UTC timestamp of the action (millisecond precision)
signature	HMAC-SHA256 signature of the audit record (Vault signing key); tamper-evident

Audit Vault Tamper Resistance

Audit records are chained: each record's signature includes the hash of the previous record's signature (Merkle-chain structure). Any tampering with a record or its sequence is detectable by re-verifying the chain. The chain root is exported weekly to an immutable external store (S3 Object Lock + customer-controlled S3 bucket optional).

5.7 Vulnerability Management

Scope	Tool	Frequency	SLA: Critical	SLA: High
Container Images	Trivy (CI/CD) + ECR scanning (prod)	Every build + daily prod scan	24h patch + redeploy	72h
Go Dependencies	govulncheck + Snyk	Every build	24h	72h
PostgreSQL / Redis / Kafka	Dependabot + manual tracking	Weekly	24h upgrade	72h
Infrastructure (IaC)	Checkov + tfsec in CI/CD pipeline	Every IaC change	24h	72h
SAST (source code)	Semgrep rules (Go + Python + TypeScript)	Every PR	Block merge + 24h fix	72h
DAST (live surface)	OWASP ZAP (weekly automated scan)	Weekly	24h + WAF rule update	72h
Secret Scanning	Trufflehog (git hooks + CI)	Every commit	Immediate rotation	24h rotation

6 SDE Runtime & Toolchain System

The SDE Runtime is the execution environment that powers every active SDE. It runs as a Kubernetes-native workload within a dedicated namespace per SDE. This section defines the runtime architecture, toolchain system internals, repository subsystem, CMS, communications module, and the manifest parsing/validation pipeline.

6.1 SDE Kubernetes Architecture

Each active SDE runs in a dedicated Kubernetes namespace. The namespace is provisioned by the SDE Provisioner during activation and follows a standardised resource template:

k8s Resource	Name Pattern	Purpose	Resource Limits
Namespace	qala-sde-{sde_id}	Tenant isolation boundary	N/A
ResourceQuota	sde-quota	Enforce CPU, memory, storage limits per scale_config	Configured from SDE scale_config
NetworkPolicy	sde-netpol	Allow intra-namespace; deny all ingress/egress from other namespaces	N/A
AuthorizationPolicy	sde-authz	Istio policy: require mTLS for all intra-namespace traffic	N/A
ServiceAccount	sde-workload-sa	Workload identity bound to Vault AppRole	N/A
ConfigMap	sde-manifest	Parsed .qala.yaml content available to all pods	N/A
Deployment	sde-api-{sde_id}	SDE API server pod (proxies solution requests)	CPU: 0.5–2, Mem: 256MB–2GB
Deployment	sde-toolchain-{toolchain_id}	One per attached toolchain; containerised tools	Per toolchain spec
Deployment	sde-repo-api-{sde_id}	Repository API proxy (read/write to object store)	CPU: 0.25–1, Mem: 128MB–512MB
Deployment	sde-cms-{sde_id}	CMS API for playbooks and solution books	CPU: 0.25–1, Mem: 128MB–512MB
HorizontalPodAutoscaler	sde-api-hpa	Scales sde-api-{sde_id} between min/max_instances per CPU/RPS	From scale_config
PersistentVolumeClaim	sde-workspace-pvc	Shared workspace volume for toolchain jobs	20GB default; configurable
CronJob	sde-health-reporter	Reports SDE health metrics to platform every 60s	CPU: 0.1, Mem: 64MB

6.2 .qala.yaml Manifest: Full Specification

The .qala.yaml manifest is the single configuration file that defines an SDE. It is parsed and validated on every mutation. Below is the complete specification with all fields, their types, defaults, and validation rules.

```
  # .qala.yaml  —  Full Specification
  sde:
    id:           string   # UUID. Assigned by platform on creation. Immutable.
    name:         string   # 3-128 chars; [a-z0-9-] after normalisation.
    version:      string   # SemVer. Auto-incremented on manifest changes.
    domain:       string   # Must match an installed domain pack.
    factory:      string   # Slug or UUID of parent factory.
    description:  string   # Optional. Max 1024 chars.
    labels:       map      # Optional arbitrary key-value metadata.

  toolchain:
    - id:         string   # Toolchain pack ID from Marketplace.
      version:    string   # Exact version or range (^1.2, ~1.2.3).
      config:     map      # Toolchain-specific configuration.
      optional:   bool     # Default false. If true, activation proceeds if
unavailable.

  repositories:
    artifacts:    string   # Repo ID (auto-created if omitted).
    assets:       string   # Repo ID.
    capital:      string   # Repo ID.
    resources:    string   # Repo ID.
    retention:
      artifacts_days: int  # 0 = indefinite. Default: 365.
      assets_days:    int  # Default: indefinite (0).

  scalability:
    min_instances:  int    # Minimum SDE API pods. Default: 1. Min: 1.
    max_instances:  int    # Maximum SDE API pods. Default: 5. Max: 100.
    cpu_limit:      string # Per-pod CPU limit. e.g. "2". Default: "1".
    memory_limit:   string # Per-pod memory limit. e.g. "2Gi". Default: "1Gi".
    workspace_size: string # PVC size. e.g. "50Gi". Default: "20Gi".
    auto_scale_policy: string  # "cpu_70pct" | "rps_100" | "none". Default:
"cpu_70pct".

  networking:
    ingress_enabled:  bool    # Expose SDE API externally. Default: false.
    ingress_hostname: string  # Custom subdomain if ingress_enabled.
    egress_rules:
      - host: string          # Allowed external destination.
        port: int
        protocol: TCP|UDP
    webhooks:
      - id:     string
        url:    string        # HTTPS URL target.
        events: [string]      # List of event types to forward.
        secret: vault:path    # HMAC signing secret from Vault.
```

```
cms:
  solution_book_template: string  # Template ID for new solution books.
  playbook_template:      string  # Template ID for new playbooks.
  default_language:       string  # ISO 639-1. Default: "en".
```

6.3 Manifest Validation Pipeline

Every manifest mutation goes through a 5-stage validation pipeline before being accepted:

6. Schema Validation — JSON Schema validation against the .qala.yaml specification. Syntax errors caught here.
7. Semantic Validation — Cross-field checks: toolchain version resolution against Marketplace; domain pack ID exists in factory; scalability values within platform limits.
8. Security Validation — Egress rules against factory allowlist policy; webhook URLs against SSRF protection list; no internal platform endpoints exposed.
9. Resource Validation — Requested CPU/memory within tenant quota; workspace size within plan limits; PVC expansion checked if existing SDE.
10. Dry-Run Provisioning — For new SDEs, a k8s namespace dry-run (--dry-run=server) verifies the manifest produces a valid ResourceQuota and NetworkPolicy before committing.

6.4 Toolchain System Internals

Toolchains are packaged as Helm charts published to the Qala Toolchain Marketplace. Each chart bundles tool binaries, configuration, and the Qala Tool Protocol (QTP) adapter that allows the SDE to communicate with the tool in a standardised way.

QTP Adapter Interface	Method	Request Type	Response Type
Execute	Run a tool invocation (e.g. run linter, compile, test)	ExecuteRequest{tool_id, command, args, input_ref, config}	ExecuteResult{exit_code, stdout_ref, stderr_ref, artefact_refs}
Validate	Validate tool configuration without executing	ValidateRequest{config}	ValidationResult{valid, errors[]}
Status	Get current tool execution status (for long-running tools)	StatusRequest{execution_id}	StatusResult{state, progress, estimated_completion}
Cancel	Cancel a running tool execution	CancelRequest{execution_id, reason}	CancelResult{cancelled, final_state}
GetArtefacts	Retrieve artefact references produced by an execution	GetArtefactsRequest{execution_id}	ArtefactsResult{artefact_refs[]}

Tool executions are asynchronous. The SDE API accepts the execute request, assigns an execution_id, queues the job, and returns 202 Accepted. The client polls /executions/{id} or subscribes to sde:{sde_id} WebSocket for the execution.completed event.

6.5 Repository Subsystem

The four repository types share a common storage backend (S3-compatible object store) but differ in access semantics, indexing, and retention policy:

Repository	Object Key Pattern	Immutability	Index	Access Control
Artifact	{tenant}/{sde}/{solution}/{version}/{artefact_id}/{filename}	Released artefacts immutable; draft mutable	Full metadata index in PostgreSQL; SHA-256 indexed in Redis bloom filter for dedup	SDE members read; toolchain write; release_manager publish
Asset	{tenant}/{sde}/{asset_type}/{asset_id}/{version}/{filename}	Version-immutable once tagged; HEAD mutable	Version history in PostgreSQL; full-text indexed in OpenSearch	SDE members read+write; factory_admin full
Capital	{tenant}/{sde}/capital/{record_id}/{filename}	Immutable (chain-of-custody)	Chain-of-custody log in PostgreSQL (append-only)	SDE owner + compliance_officer read; root_admin only write
Resource	{tenant}/{sde}/resources/{type}/{resource_id}	Mutable with version history	Resource graph in PostgreSQL (adjacency list)	SDE owner + factory_admin write; SDE members read

6.6 CMS: Playbook & Solution Book Engine

The CMS is a content management layer built on top of the Asset Repository. It provides structured document management, versioning, template rendering, and auto-population.

Feature	Design
Document Model	Documents are stored as versioned JSON documents conforming to a section schema. Sections are typed: text, table, diagram, code, metric. Rendered to HTML/Markdown/PDF on demand.
Version Control	Every document save creates a new version. Version history is navigable. Diff is computed between any two versions at the section level.
Auto-Population	Designated sections are auto-populated from platform data: changelog from CCR history; quality gate results from Release Service; member list from RBAC Service; artefact manifest from Artifact Repository.

Template Engine	Templates define the section structure for new playbooks and solution books. Domain packs can provide domain-specific templates (e.g. a Pharma domain pack provides a GxP-compliant solution book template).
AI Assist	The AI Agent service can generate draft section content based on solution context. Output is placed in a DRAFT layer within the section; human review and approval promotes it to ACCEPTED.
Export	Documents export to PDF (via headless Chrome), DOCX (via pandoc), and Markdown. Export is a versioned, indexed artefact placed in the Artifact Repository.
Search	All CMS content is indexed in OpenSearch. Full-text search across playbooks and solution books is available within factory scope.

7 AI Agent Design

The Qala AI Agent is a system-level intelligence layer that continuously monitors all governed entities, provides proactive recommendations, performs drift detection, assists document authoring, and computes risk signals. The AI Agent is a read-only consumer of all domain events and writes only to designated AI output fields — it never executes state transitions directly.

7.1 AI Agent Architecture

The AI Agent is a Python 3.12 service with a modular capability architecture. Each capability is an independent worker that consumes a subset of domain events and produces AI outputs.

Attribute	Value
Service Name	qala-ai-agent
Language	Python 3.12 + FastAPI (control plane) + asyncio workers (capability workers)
Model Backend	Anthropic Claude API (claude-sonnet-4-20250514 for generation; claude-haiku for classification)
Event Consumption	Kafka consumer group: qala-ai-agent, consuming all qala.*.events.v1 topics
Output Targets	Writes to PostgreSQL ai_outputs table; publishes ai.recommendation.created events to Kafka
Rate Limiting	LLM API calls rate-limited to 100 RPM; queued via internal priority queue
Tenant Isolation	AI Agent enforces tenant scoping on all reads. LLM prompts never include cross-tenant data.

7.2 AI Capabilities

Capability	Trigger	Input Context	Output	Model
Solution Risk Analysis	solution.state_transitioned, solution.version_published	Solution fields, CCR history, quality gate results, domain pack risk profile.	Risk score update (1–10), risk rationale, top 3 risk factors.	claude-haiku (classification)
Drift Detection	Scheduled: daily per active solution	Current solution state vs. last-released state; time since last CCR.	Drift flag (none/minor/major/critical), drift description, recommended actions.	claude-haiku (classification)
CCR Impact Analysis	ccr.submitted	CCR description, affected solutions, their current states and release history.	Computed secondary impact list (solutions not in submitted impact scope that may be affected); confidence.	claude-sonnet-4-20250514

Playbook Section Drafting	User-triggered via /solutions/{id}/ai/draft-playbook-section	Solution metadata, SDE config, existing playbook sections, domain pack schema.	Draft text for requested playbook section (ADR, architecture description, etc.).	claude-sonnet-4-20250514
Release Notes Generation	release.published	All CCRs implemented in this release; solution changelog; diff from previous version.	Formatted release notes in Markdown.	claude-sonnet-4-20250514
Quality Gate Interpretation	release.gate_failed	Gate failure findings, solution codebase (if software domain), similar historical failures.	Plain-English explanation of failure + recommended remediation steps.	claude-sonnet-4-20250514
Anomaly Detection	Scheduled: hourly audit log scan per factory	Audit event patterns for last 24h vs. historical baseline.	Anomaly flag + event IDs flagged; severity (low/medium/high/critical); description.	Isolation Forest (local model) + claude-haiku (enrichment)
Solution Book Auto-Update	solution.state_transitioned, release.published	New solution state context; auto-populated section schemas.	Updated values for auto-populated sections (changelog, status, version, last release date).	Rule-based (no LLM for deterministic fields)

7.3 Prompt Architecture & Tenant Safety

All LLM-based capabilities follow a structured prompt template. Tenant data isolation is enforced at the prompt level:

AI Prompt Safety Rules (binding)

1. NEVER include data from more than one tenant in a single prompt.

2. ALWAYS include the tenant_id and solution_id in the system prompt as context bounds.

3. NEVER include secret field values (API keys, passwords, credentials) in prompts.

4. ALWAYS instruct the model to respond in structured JSON format for machine-parsed outputs.

5. ALWAYS apply output validation: if parsed output fails schema validation, discard and log; do NOT write invalid AI output to the DB.

6. Prompts are versioned alongside the capability code. Prompt changes require review (prompt engineering review gate in CI/CD).

7.4 AI Output Schema

All AI outputs are stored in a standardised schema in the ai_outputs table, linked to the entity they describe:

```
CREATE TABLE ai_outputs (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id       UUID NOT NULL,
  capability      TEXT NOT NULL,     -- e.g. "solution_risk_analysis"
  entity_type     TEXT NOT NULL,     -- e.g. "Solution", "CCR", "Release"
  entity_id       UUID NOT NULL,
  output          JSONB NOT NULL,    -- Validated output blob
```

```
  model_id        TEXT NOT NULL,     -- e.g. "claude-sonnet-4-20250514"
  prompt_version  TEXT NOT NULL,
  confidence      FLOAT,            -- If applicable
  generated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  expires_at      TIMESTAMPTZ,      -- For time-limited analyses
  is_stale        BOOL DEFAULT FALSE -- Set when entity changes after generation
);
```

8 Domain Pack System

Domain Packs are the extensibility mechanism that makes Qala universally applicable. This section defines the full Domain Pack specification: schema extension, lifecycle state customisation, quality gate definition, CCR type configuration, UI vocabulary adaptation, and the Domain Pack publication/installation workflow.

8.1 Domain Pack Specification (Full)

A Domain Pack is a JSON document that extends the Qala Universal Solution Schema. It is versioned using SemVer, cryptographically signed by its publisher, and validated against the Domain Pack Schema before installation.

Field	Type	Required	Description
id	string (slug)	Yes	Globally unique identifier: publisher-domain-version, e.g. qala-software-v1
name	string	Yes	Human-readable name. Max 128 chars.
version	semver string	Yes	SemVer. Patch = backward-compatible field additions. Minor = new states/gates. Major = breaking changes.
domain	string (enum)	Yes	Canonical domain key: software, cpg, pharma, financial, agricultural, legal, tax, research, generic.
publisher	string	Yes	Publisher org ID in Qala Marketplace. Verified publishers show a checkmark.
extends	string	Yes	Base schema reference: always "qala/base-solution-schema/v1" unless extending another domain pack.
fields	FieldDef[]	No	Additional fields added to the Universal Solution Schema for solutions using this domain.
lifecycle_states	LifecycleDef	No	null = inherit universal lifecycle. Set to override or extend with domain-specific states.
quality_gates	GateDef[]	No	Named evaluation gates with evaluator ID, required_for_release flag, and config schema.
ccr_types	string[]	No	CCR types valid for this domain. Additional types on top of the universal set.
risk_modifiers	RiskModifiers	No	domain_base_risk: float (1-3); regulatory_flag: bool; custom_risk_factors: RiskFactor[].
ui_vocabulary	VocabularyDef	No	Vocabulary overrides for platform UI labels in this domain context.
templates	TemplateDef[]	No	Playbook and Solution Book templates to pre-install in factories using this domain pack.
evaluators	EvaluatorDef[]	No	Container image refs for quality gate evaluators provided by this pack.

8.2 FieldDef Specification

Field Property	Type	Description
name	string	Field name (snake_case). Must not clash with universal schema field names.
label	string	Human-readable label for UI display.
type	enum	string | number | boolean | date | datetime | enum | array<string> | json
enum	string[]	For type=enum: allowed values.
pattern	regex string	For type=string: validation regex.
min/max	number	For type=number: inclusive bounds.
required	boolean	If true, solution cannot enter In Review without this field set.
required_for	string[]	List of lifecycle states that require this field. More granular than required.
searchable	boolean	If true, field is indexed in OpenSearch for faceted search.
pii	boolean	If true, field is excluded from AI prompts and audit export by default.
encrypted	boolean	If true, field is encrypted at-rest with tenant key before storage in JSONB.

8.3 Built-In Domain Packs

Qala ships with the following built-in domain packs maintained by the Qala Domain Team:

Domain Pack	Version	Domain-Specific States	Quality Gates	Key Fields
qala-software-v1	1.2.0	Inherited universal	SAST, Coverage, Dependency Check, SLSA Attestation	language, runtime, test_coverage, sast_passed, sbom_ref
qala-pharma-v1	1.1.0	+ Batch Review, QP Release, Recall	Batch Compilation, QP Sign-off, GMP Compliance, Stability	batch_number, batch_type, regulatory_filings, stability_months, qp_signature
qala-cpg-v1	1.0.0	+ Formulation Review, Batch Release	INCI Compliance, Safety Data Sheet, Regulatory Pre-check	inci_names, batch_type, eu_reach, safety_data_sheet_version
qala-financial-v1	1.1.0	+ Mandate Review, Backtest, Live Gate	Risk Gate (VaR/Sharpe), Mandate Compliance, Regulatory Reg	strategy_type, asset_class, sharpe_ratio, var_99, mandate_id, mifid_registered

qala-agricultural-v1	1.0.0	+ Field Trial, Seasonal Gate	Agronomic Gate, Traceability Gate, Seasonal Gate	crop_type, field_gps_polygon, treatment_protocol, yield_target_tha, epcis_ref
qala-legal-v1	1.0.0	+ Legal Review, Effective-Date Gate	Legal Review, Conflict Check, Effective-Date Gate	jurisdiction_code, effective_date, legislative_ref, filing_status, counsel_sign
qala-tax-v1	1.0.0	+ Rule Validation, Activation Gate	Rule Integrity Gate, Effective-Date Gate	jurisdiction_code, tax_type, effective_year, rule_set_version
qala-research-v1	1.0.0	+ Peer Review, Reproducibility Gate	Peer Review Gate, Reproducibility Gate, IRB Approval Gate	hypothesis, experimental_desig n, dataset_version, irb_approved, preregistration_doi

8.4 Domain Pack Installation Lifecycle

Installing a Domain Pack into a factory is a governed operation. The factory admin installs the pack; the installation triggers validation, compatibility checks, and model revalidation:

11. Admin submits InstallDomainPack command: {factory_id, pack_id, pack_version}.
12. Pack Registry Service fetches the pack manifest and verifies cryptographic signature against publisher certificate.
13. Compatibility Check: verifies the pack version is compatible with all existing domain packs in the factory (no field name conflicts, no lifecycle state conflicts).
14. Dry-Run Revalidation: all existing SolutionModel definitions in the factory are validated against the new merged schema. Any validation errors surface as a pre-installation report.
15. On confirmation: factory.domain_pack_attached event published; pack added to factory's active pack list; SDEs in the domain can now use the new fields and gates.
16. Model updates: if new fields are required_for=[In Review], existing solutions in Draft state remain valid; the requirement only applies on next state transition.

9 Infrastructure & Deployment Architecture

Qala is deployed on Kubernetes and designed to run on any CNCF-conformant cluster. This section covers the production deployment topology, IaC strategy, multi-region architecture, DR/backup systems, and the CI/CD pipeline that governs platform deployments. The platform governs its own releases using itself (the root factory principle in practice).

9.1 Production Deployment Topology (SaaS)

The SaaS deployment spans three active regions (us-east-1, eu-west-1, ap-southeast-1) with a global control plane and regional data planes:

Component	Region Placement	Replication Strategy	Failover
API Gateway	All 3 regions (active-active)	Stateless; traffic split by Anycast DNS (latency-based)	DNS failover < 30s; session tokens valid cross-region
Domain Services	All 3 regions (active-active)	Stateless; command routing via global Kafka topic	Service restart < 60s; circuit breaker auto-failover
PostgreSQL (primary)	1 primary per region	Patroni HA cluster; 3 replicas per region; cross-region WAL streaming	Patroni automatic failover < 30s; cross-region promoted in 5min
Kafka	All 3 regions	KRaft cluster per region; MirrorMaker2 for cross-region replication	Regional Kafka failure: consumers switch to mirrored topic < 60s
Redis	All 3 regions	Redis Cluster per region; no cross-region replication (cache-only)	Redis failure: cold start from DB; latency spike for 2-5 min
Object Storage	Global (S3 + multi-region CRR)	S3 Cross-Region Replication; versioning enabled; Object Lock for releases	Cross-region replication; no data loss
OpenSearch	1 cluster per region	Periodic full re-index from DB; not a system of record	Re-index from DB; latency spike in search during rebuild
Vault	All 3 regions (raft cluster)	Raft consensus cluster across regions	Raft automatic leader election < 10s

9.2 Infrastructure as Code

All Qala infrastructure is defined as code with zero manual provisioning. The IaC stack uses Terraform for cloud resources and Helm + Kustomize for Kubernetes workloads:

Layer	Tool	Repository	Environment Management
Cloud Infrastructure	Terraform (HCL)	qala-infra-cloud	Workspaces per environment (dev, staging, prod-{region}). State in S3 + DynamoDB lock.
Kubernetes Platform	Helm + Helmfile	qala-infra-k8s	Helmfile environments; values overlaid per environment. ArgoCD for GitOps sync.
Platform Services	Kustomize + ArgoCD	qala-platform-config	ArgoCD ApplicationSet generates per-environment manifests from base + overlay pattern.
SDE Provisioning	Operator (Go)	qala-sde-operator	Custom k8s operator reconciles SDE CRDs to desired state. Managed by platform services, not IaC directly.
Secrets	Vault Terraform provider + External Secrets Operator	qala-infra-secrets	Vault policies and mounts via Terraform. k8s Secrets synced by ESO.
DNS + TLS	Terraform (Route53/CloudFlare) + cert-manager	qala-infra-network	Wildcard certs for *.sde.qala.io per region; tenant subdomains auto-provisioned.

9.3 CI/CD Pipeline (Platform Self-Governance)

Platform releases are governed through Qala itself. The Qala Engineering factory is a factory on the platform. Every platform component is a solution. Release to production requires a CCR and a platform release with quality gates.

Pipeline Stage	Tool	Trigger	Quality Gate	On Failure
1. Build	GitHub Actions + ko (Go container builder)	PR or main push	Container image builds without errors	Block PR merge
2. Test (Unit)	go test -race -cover	Every commit	Coverage ≥80%; no race conditions; all tests pass	Block PR merge
3. Test (Integration)	Docker Compose test environment	PR merge to main	All integration tests pass; no DB schema drift	Revert merge; alert on-call
4. SAST	Semgrep (Go ruleset)	Every commit	No Critical or High findings	Block PR merge

5. Container Scan	Trivy	Every build	No Critical CVEs in container image	Block PR merge; update base image if CVE in base
6. Schema Lint	Buf CLI (Protobuf)	Protobuf changes only	No breaking changes to gRPC API surface	Block PR merge
7. Staging Deploy	ArgoCD (automated)	Merge to main	E2E synthetic tests pass (Playwright)	Auto-rollb ack; alert on-call
8. CCR + Approval	Qala platform (CCR workflow)	Staging validated; release ready	CCR approved by Engineering Lead + QA Lead (Medium risk)	CCR rejected; fix required
9. Production Deploy	ArgoCD (manual sync after CCR approved)	CCR approved	Production healthchecks pass; error rate <0.1% for 5 min	Auto-rollb ack; PagerDuty page

9.4 Backup & Disaster Recovery

Asset	Backup Method	Frequency	RPO Target	RTO Target	Restore Procedure
PostgreSQL	pg_basebackup + continuous WAL archiving to S3	Continuous (WAL) + daily base	<1 min	<15 min	Patroni PITR restore; automated runbook
Kafka Topics	MirrorMaker2 cross-region replication + S3 tiered storage	Continuous	<30 sec	<5 min	Consumer group reset to earliest offset; replay from regional mirror
Object Storage	S3 Cross-Region Replication + S3 Object Lock (releases)	Continuous	~0 sec	<5 min	Cross-region bucket promoted; DNS updated
Vault	Vault Raft snapshots to S3 (encrypted)	Hourly	<1 hour	<30 min	Restore from snapshot; unseal with shamir keys
Redis	No backup (cache only). Cold-start from DB projections.	N/A	N/A (cache)	<10 min (rebuild)	Flush + warm from DB
OpenSearch	Snapshot to S3 (periodic) + rebuild from Kafka replay	Daily snapshot + on-demand	<24 hours (snp) / full rebuild	<2 hours	Restore from snapshot; incremental replay to current

9.5 Multi-Tenant Data Residency

Enterprise tenants can select their data residency region at factory creation time. Data residency guarantees are enforced by the following mechanisms:

- Factory namespace created exclusively in the selected region's Kubernetes cluster and PostgreSQL instance.
- Kafka events for the tenant are produced to and consumed from the regional cluster only. MirrorMaker2 does not replicate tenant-scoped topics cross-region without explicit opt-in.
- Object Storage buckets are provisioned in the tenant's selected region with cross-region replication disabled unless the tenant enables it (Geo-Redundancy add-on).
- The global control plane stores only tenant metadata (factory IDs, member IDs, plan info) — no solution content or event payloads cross regional boundaries.
- Egress network policies on SDE pods enforce outbound traffic only to approved endpoints in the tenant's selected region.

10 Observability Design

Every Qala service is instrumented with OpenTelemetry. Metrics, traces, and logs flow to a unified observability stack. This section defines the instrumentation standards, key SLOs, alerting rules, and on-call runbook structure.

10.1 Instrumentation Standards

All Qala services must implement the following instrumentation contract:

Signal	Standard	Key Requirements
Traces	OpenTelemetry SDK (Go otelhttp + otelsql)	Every incoming HTTP request and outgoing HTTP/gRPC call gets a span. DB queries instrumented. Kafka produce/consume instrumented. Span names: "{service}.{operation}" e.g. "sde-service.ActivateSDE". Baggage: tenant_id, factory_id, correlation_id.
Metrics	Prometheus + OpenTelemetry Metrics SDK	Request rate (reqs/sec per endpoint), error rate (%), P50/P95/P99 latency, queue depth (Kafka consumer lag), DB connection pool utilisation, Redis cache hit ratio. Service-specific business metrics (e.g. sde_activations_total).
Logs	Structured JSON logs to stdout (Loki scrape)	Level: INFO/WARN/ERROR/DEBUG. All logs include: timestamp, service, version, tenant_id, correlation_id, trace_id, span_id. Errors include full stack trace. No PII in log lines (sanitised at log site).

10.2 Key Service Level Objectives (SLOs)

SLO	Target	Measurement Window	Alert Threshold	Burn Rate
API Availability (all endpoints)	99.9%	30-day rolling	Error budget burn > 5x (15-min window)	Page on-call
API P99 Latency (read)	<80ms	5-min rolling	P99 > 80ms for 10 consecutive minutes	Slack alert
API P99 Latency (write/command)	<200ms	5-min rolling	P99 > 200ms for 10 consecutive minutes	Slack alert
SDE Activation Time	<30s	1-hour rolling	P95 > 30s for 5 consecutive activations	Slack alert + ticket
CCR Notification Delivery	<30s	15-min rolling	P99 > 30s	Slack alert
Event Processing Lag (Kafka)	<500ms	5-min rolling	Consumer lag > 10,000 messages	Slack alert + auto-scale
Audit Event Write	99.99%	30-day rolling	ANY audit event write failure	Page on-call

				immediately
AI Agent Recommendation Latency	<60s	15-min rolling	P95 > 60s (non-blocking; advisory only)	Slack alert

10.3 Alerting Architecture

Alerts are generated by Prometheus AlertManager and routed by severity and service ownership. All alerts have a defined runbook URL embedded in the alert labels.

Severity	Definition	Routing	Response SLA
Critical (P1)	Production data loss risk or full service unavailability.	PagerDuty immediate page to on-call engineer + incident channel created	<15 min acknowledge, <1h mitigation
High (P2)	Degraded service (elevated error rate, latency SLO breach).	PagerDuty page during business hours; Slack alert after hours	<30 min acknowledge, <4h mitigation
Medium (P3)	Elevated warning, approaching SLO threshold, non-user-facing service issue.	Slack alert (#platform-alerts)	<4h acknowledge, <24h resolution
Low (P4)	Advisory signal, trend, or informational anomaly.	Slack alert (#platform-info)	Next business day

10.4 Distributed Tracing Strategy

Every user-initiated API request generates a trace that spans the full call path from API Gateway to database. Traces are sampled at 100% for errors and P99+ latency outliers; 10% base sampling for nominal traffic. Trace data is stored in Tempo (30-day retention for prod; 7-day for staging).

- Trace ID is included in all API error responses as the incident ID, enabling support teams to correlate user-reported errors to traces.
- Cross-service traces are linked via Kafka header propagation (traceparent W3C TraceContext header in Kafka message headers).
- Database traces show full SQL query text (redacted for PII fields) and row counts.
- AI Agent traces include the prompt template version and model ID, enabling reproducibility analysis.

11 Integration Architecture

Qala's integration architecture supports three integration patterns: pull-based (the external system queries Qala via API), push-based (Qala sends events to external systems via webhooks), and adapter-based (Qala natively integrates with domain-specific external systems via Domain Integration Adapters). This section defines all three patterns in detail.

11.1 Native Integrations (Built-In)

Integration	Type	Capabilities	Auth Method
GitHub	Adapter	Link GitHub repos to Asset Repository; trigger SDE toolchain on push; import commit history as solution changelog; auto-populate solution fields from repo metadata.	GitHub App installation (fine-grained permissions)
GitLab	Adapter	Same as GitHub. GitLab CI/CD pipeline triggers mapped to Qala toolchain executions.	GitLab OAuth App + group access token
Jira	Adapter	Link Jira issues to CCRs; sync CCR status to Jira issue status; create Jira issues from AI recommendations.	Jira OAuth 2.0 (3LO)
Confluence	Adapter	Sync Solution Book pages to Confluence spaces; publish playbooks as Confluence pages.	Confluence OAuth 2.0 (3LO)
Slack	Push	CCR approval notifications; release published notifications; AI recommendation alerts; daily digest.	Slack App (Bot token)
Microsoft Teams	Push	Same as Slack but via Teams incoming webhook or Bot Framework.	Teams Incoming Webhook URL
ServiceNow	Adapter	Sync CCRs to ServiceNow Change Requests; two-way status sync; compliance evidence export.	ServiceNow OAuth 2.0 + Table API
AWS CodePipeline	Adapter	Trigger SDE toolchain from CodePipeline stage; push pipeline results as quality gate evidence.	AWS IAM Role assumption (OIDC)
Azure DevOps	Adapter	Sync Azure Pipelines with SDE toolchain; link Azure Repos to Asset Repository.	Azure AD App Registration + DevOps PAT
Snyk	Evaluator	Quality gate evaluator: Dependency vulnerability scan. Results formatted as gate findings.	Snyk API Token (stored in Vault)
LIMS (Benchling, etc)	Adapter	For Pharma/CPG domain: sync formulation records, batch data, regulatory submissions to Qala solution fields.	API key per integration (Vault-stored)
Bloomberg Terminal	Adapter	For Financial domain: pull security data, risk metrics into solution fields.	Bloomberg B-PIPE session (enterprise only)

11.2 Webhook System Design

The Webhook Engine is a dedicated service that processes outbound event delivery. It ensures at-least-once delivery with exponential backoff retry and a dead-letter queue for persistent failures.

Attribute	Value
Service	qala-webhook-engine (Go)
Delivery Guarantee	At-least-once. Idempotency via X-Qala-Event-ID header (UUID v7 per delivery attempt).
Retry Policy	Exponential backoff: 10s, 30s, 2min, 10min, 1h, 6h, 24h. 7 attempts max.
Timeout	Request timeout: 10s per attempt.
Dead Letter	After 7 failures: move to webhook_dead_letter queue; alert SDE owner; visible in SDE UI.
Signature	X-Qala-Signature header: HMAC-SHA256(secret, payload). Secret from Vault.
Payload Format	Standard event envelope (Section 2.2) + webhook_id metadata.
Fan-out	One event can trigger multiple webhooks (one per SDE subscription). Delivered in parallel.
Rate Limiting	100 deliveries/min per webhook endpoint. Excess queued.

11.3 Domain Integration Adapter Protocol

Domain Integration Adapters are Go plugins (compiled as separate binaries) that implement the Qala Integration Adapter Protocol (QIAP). Adapters run in the SDE namespace and are attached via the SDE manifest.

```go
// QIAP Interface (Go)
type DomainAdapter interface {
  Name()    string
  Version() string
  Domain()  string  // e.g. "pharma", "financial"

  // Capabilities this adapter provides
  Capabilities() []Capability  // "field_sync", "artefact_import", "event_forward"

  // Called when a solution field changes; adapter can push to external system
  OnFieldUpdate(ctx context.Context, e FieldUpdateEvent) error

  // Called to pull data from external system into solution fields
  PullData(ctx context.Context, solutionID uuid.UUID) (map[string]any, error)

  // Called to import an artefact from external system into Asset Repository
  ImportArtefact(ctx context.Context, req ArtefactImportRequest) (*Artefact, error)
}
```

11.4 Import & Export

Qala supports bulk import and export of all managed entities. Export is used for: migration between instances, backup portability, marketplace publishing, and compliance evidence packaging.

Export Type	Format	Contents	Encryption
Factory Export	ZIP containing JSON manifests	Factory metadata, policy, member list (no personal data), SDE manifest refs.	AES-256 with tenant key. Passphrase-protected ZIP option.
SDE Export	OCI artefact (qala/sde bundle)	Full .qala.yaml, toolchain list, repository index, CMS document index (not content).	OCI layer encryption; Cosign-signed manifest.
Solution Export	ZIP containing JSON + artefacts	Full solution schema (all versions), CCR history, release history, artefact bundle.	AES-256 with tenant key.
Audit Package Export	ZIP containing JSONL + signature chain	All audit events for a time range; HMAC signature chain; verification script.	AES-256 + chain verification key included.
Compliance Package	ZIP containing PDF reports + evidence JSON	Quality gate results, CCR audit, release attestations, domain-specific compliance evidence.	AES-256 with tenant key.

12 Marketplace Architecture

The Qala Marketplace is a governed registry of reusable platform components: Domain Packs, Toolchains, Factory Blueprints, Playbook Templates, and Solution Books. It enables the Qala ecosystem to grow without platform core changes, and allows enterprise organisations to publish internal components for reuse across their factories.

12.1 Marketplace Component Types

Component Type	Publisher	Consumer	Version Scheme	Distribution Mechanism
Domain Pack	Qala (built-in) + verified publishers	Factory Admins (install into factory)	SemVer	JSON document served via Pack Registry API; schema signed with publisher certificate
Toolchain	Qala + community + enterprise	SDE Owners (attach to SDE)	SemVer	Helm chart in OCI registry (oci://registry.qala.io/toolchains/{name})
Factory Blueprint	Qala + enterprise (internal registry)	Factory Admins (instantiate new factory)	SemVer	ZIP bundle (factory manifest JSON + domain pack refs + SDE template refs)
Playbook Template	Qala + domain packs + community	SDE members (create new playbooks)	SemVer	JSON template document in CMS template registry
Solution Book Template	Qala + domain packs + community	SDE members (create new solution books)	SemVer	JSON template document in CMS template registry
Quality Gate Evaluator	Qala + domain packs + enterprise	Domain Packs (reference evaluator container)	SemVer	Container image in OCI registry; SHA-256 pinned in domain pack definition

12.2 Publishing Workflow

Publishing to the Qala Marketplace is governed by a review and approval workflow. This ensures quality and security of all components available to the ecosystem.

17. Publisher submits component via Marketplace Publisher Portal or qala marketplace publish CLI command.
18. Automated validation: schema validation, security scan (container image scan for evaluators; JSON schema lint for domain packs), licence compatibility check.
19. Automated smoke test: component is installed in a sandboxed test factory; basic functionality verified.

20. Human review (for public marketplace): Qala Marketplace team reviews for quality, documentation completeness, and policy compliance.
21. Publisher signs the component with their verified Sigstore certificate. Qala counter-signs on approval.
22. Component published to registry; available for installation; indexed in Marketplace search.

Verified Publisher Programme

Publishers can apply for Verified Publisher status (similar to GitHub Verified). Verified publishers' components show a checkmark badge. Verification requires: legal entity confirmation, security review, at least 3 months of published components with no security incidents, and agreement to Marketplace Trust Policy.

12.3 Registry Technical Design

Attribute	Value
Domain Packs	Stored in PostgreSQL marketplace.packs table (JSON). Served via Pack Registry REST API. Cached in Redis (TTL: 5min).
Toolchains (Helm OCI)	Stored in dedicated OCI registry (Harbor or GHCR-compatible). Pull via standard Helm CLI. Integrity: SHA-256 content-addressed.
Container Images (Evaluators)	OCI registry with namespaced repositories per publisher. Cosign signature mandatory for all published evaluator images. Trivy scan required.
Marketplace Search	OpenSearch index. Indexed fields: name, description, domain, tags, publisher, version, download_count, last_updated. Faceted filters by domain and type.
Rate Limiting	Marketplace API: 1,000 reads/min/tenant; 10 publish operations/day/publisher. Registry pull: governed by OCI registry pull-through cache.

13 Performance Design & Capacity Planning

This section defines the performance model, caching architecture, database optimisation strategy, and capacity planning methodology. Performance targets are derived from the NFRs in Section 1.6 and validated through load testing.

13.1 Caching Architecture

Cache Layer	Technology	Cache Keys Pattern	TTL Strategy	Invalidation
API Response Cache	Redis (cluster)	tenant:{tid}:solution:{id}:detail	60s (volatile entities), 300s (stable entities like models)	Event-driven: on doma event, delete affected keys
Permission Cache	Redis (cluster)	tenant:{tid}:user:{uid}:perms	120s	Invalidate on rbac.role_changed ev
Factory Policy Cache	Redis (cluster)	tenant:{tid}:factory:{fid}:policy	300s	Invalidate on factory.policy_updated event
JWKS Public Keys	In-process (Go)	N/A (singleton)	Refreshed every 300s; on JWT validation failure (forced)	Background goroutine staleness tolerated up 5min
Domain Pack Cache	Redis + in-process	pack:{pack_id}:{version}	Indefinite (immutable versions never change)	Only when new versio published; old version keys persist
Search Suggestions	Redis (cluster)	tenant:{tid}:search:suggest:{prefix}	30s	Not invalidated; TTL expiry tolerated
Projection Warm Cache	Redis (cluster)	tenant:{tid}:projection:{type}:page:{n}	10s (hot pages), 60s (cold pages)	Event-driven invalidati for affected tenant+typ

13.2 Database Query Optimisation

The following critical query patterns have been optimised with composite indexes and query design patterns:

Query Pattern	Table	Index	Notes
List solutions by SDE (paginated by updated_at)	solutions	CREATE INDEX idx_sol_sde_updated ON solutions(sde_id, updated_at DESC) WHERE state != 'retired'	Partial index excludes retired solutions; improves active solution list performance by ~10x
List open CCRs for approver	ccrs	CREATE INDEX idx_ccr_approver ON ccrs USING GIN(approver_ids)	GIN index on approver_ids

			array; supports @> operator for "contains approver" check
Factory hierarchy traversal	factories	ltree column path indexed with GiST index	Hierarchical queries use ltree ancestor/descendant operators; avoids recursive CTE
Audit query by actor + time range	audit_events	CREATE INDEX idx_audit_actor_time ON audit_events(tenant_id, actor_id, occurred_at DESC)	Supports compliance officer audit queries: "all actions by user X in date range"
Event replay by aggregate (ordering)	events	CREATE INDEX idx_events_aggregate ON events(aggregate_id, occurred_at ASC)	Used exclusively by event replay during projection rebuild
Search solutions by tags + domain	solutions	CREATE INDEX idx_sol_tags ON solutions USING GIN(tags) + partial index on domain	Faceted tag filtering; GIN index for @> array contains

13.3 Capacity Planning Model

Platform capacity is modelled on a per-tenant basis and aggregated. The following sizing model is used for infrastructure capacity planning:

Resource	Formula	Assumptions	Example (10k solutions/day tenant)
API Pods	max(2, ceil(peak_rps / 200))	Each pod handles 200 RPS at P99 <80ms	peak_rps=100 → 1 pod (min 2)
Kafka Partitions	ceil(domain_events_per_sec / 1000)	1000 events/sec per partition sustainable	5 events/sec → 1 partition (min 3)
PostgreSQL IOPS	writes/sec × 3 (write amplification)	WAL + index + heap; Citus shards reduce per-node IOPS	100 writes/sec → 300 IOPS on primary
Object Storage	artefact_size_avg × artefacts_per_release × releases_per_day × retention_days	Immutable artefacts; no deduplication	50MB avg × 5 × 3 × 365 = 274GB/year
Redis Memory	(active_solutions × 4KB) + (active_sessions × 2KB)	Average JSON projection size 4KB; session token 2KB	5,000 solutions = 20MB; 1,000 sessions = 2MB
AI Agent (LLM)	ceil(llm_calls_per_hour / rate_limit)	Anthropic API 100 RPM limit; batching not available	50 events/hour triggering AI = 50 calls/hour → within limit

14 Migration, Versioning & Schema Evolution

Qala is a long-lived platform. This section defines how schema changes, API breaking changes, database migrations, and event schema evolution are managed safely across running production systems and existing tenant data.

14.1 Database Migration Strategy

All database schema changes are managed through Atlas (terraform-like declarative schema tool) with a strict no-downtime migration policy:

Migration Type	Policy	Tooling	Deployment Method
Add nullable column	Safe: zero-downtime. Deploy any time.	Atlas apply (additive only)	Rolled out in a single Atlas migration file; applied before service deployment.
Add non-nullable column	Multi-step: (1) Add nullable + default, (2) Backfill, (3) Set NOT NULL.	Atlas + custom backfill job	Three separate deployments; backfill job runs async before step 3.
Rename column	Never rename. Add new column + backfill + deprecate old.	Atlas (additive) + code update	Old column kept for 2 release cycles; removed in third release.
Add index	Safe but blocking on large tables. Use CONCURRENTLY.	Atlas (CREATE INDEX CONCURRENTLY)	Applied in off-peak window; monitored for table lock duration.
Drop column	Multi-step: (1) Remove all code references, (2) Wait 2 deployments, (3) Drop.	Atlas drop + code freeze	Coordinated deprecation cycle; minimum 2-week window.
Add new table	Safe: zero-downtime. Deploy any time.	Atlas apply	New tables always non-blocking.
Alter column type	Complex. Always via new column + migration. Breaking. Requires RFC.	Custom migration job	RFC required; tenant data migration plan required.

14.2 Event Schema Evolution

Domain event schemas evolve using an append-only strategy. Backward and forward compatibility is maintained to support in-flight event replay during rolling deployments:

Change Type	Allowed	Strategy
Add optional field to payload	Yes	New consumers read new field if present; old consumers ignore unknown fields. No version bump.

Remove field from payload	No	Fields may never be removed from published event schemas. Deprecated fields are kept with null values.
Change field type	No	Field types are immutable in a given event_version. Bump event_version and run consumers that handle both versions.
Add new event type	Yes	New consumers subscribe; old consumers ignore. Register in event catalogue. No replay impact.
Rename event type	No	Publish both old and new types for 2 release cycles; migrate consumers; deprecate old type.
Bump event_version (schema major change)	Yes (with process)	Old and new versions coexist. Projection handlers must handle both versions. Old version tombstoned after all consumers upgraded.

14.3 API Deprecation Process

23. Deprecation decision documented in an RFC. Engineering + API consumers notified 6 months in advance.
24. Deprecated endpoint marked with Sunset header (RFC 8594): Sunset: Sat, 14 Jun 2026 23:59:59 GMT.
25. Deprecation warning logged for every call to deprecated endpoint (for consumer audit).
26. Documentation updated with migration guide and new endpoint equivalent.
27. 4 months before sunset: automated emails sent to all tenants who have called the deprecated endpoint in the last 30 days.
28. 1 month before sunset: API Gateway returns 301 redirect to new endpoint (where semantically equivalent) or 410 Gone.
29. Sunset date: endpoint removed. Any remaining consumers receive clear error with migration guide URL.

14.4 Tenant Data Migration (Cross-Version)

When platform schema changes require transforming existing tenant data (e.g. new required field with default value logic that requires computed backfill), the Tenant Migration Engine handles the migration:

- Migrations are idempotent and re-runnable. The engine tracks migration state per tenant in a migrations table.
- Migrations run asynchronously post-deployment. Tenants on old schema version continue to operate in a compatibility mode.
- Migration progress is visible to tenant admins in the Platform Health section of the factory dashboard.
- For large tenants (>100k solutions), migrations are batched in 1,000-solution chunks with configurable concurrency limits to avoid DB pressure.
- Tenants can trigger migration manually or it auto-triggers after 72h of deployment.

End of Document — Qala Platform Low-Level Technical Design v1.0

QALA-LLD-001 · Confidential · Internal Engineering